

0.69. Chercher le A sur l'écran : lire

Le 0.68, plus une recherche : quand on appuie sur A, le programme regarde chaque case de l'écran avec `lire()`, et affiche où il a trouvé la lettre.

1. Ce qu'il faut comprendre

C'est le 0.68, plus un bloc : quand on appuie sur le bouton **A**, le programme **cherche** la lettre sur l'écran, case par case, et affiche où il l'a trouvée.

Ce qui est nouveau ici : `lire(colonne, ligne)`. Elle **rend la tuile affichée** dans une case de l'écran. `lire(9, 8) == ALPHABET[0]` est vrai si la case (9, 8) montre un A.

La recherche : deux boucles `for`, l'une dans l'autre, passent sur **toutes les cases** : `l` fait les 18 lignes (0 à 17), et pour chaque ligne, `c` fait les 20 colonnes (0 à 19). Soit $18 \times 20 = 360$ cases. Quand une case contient le A, on retient sa colonne dans `trouveX` et sa ligne dans `trouveY`.

Deux façons de connaître la position. `x` et `y` : le programme la **retient** lui-même, à chaque pas. `lire()` : il la **retrouve** en regardant l'écran, sans rien avoir retenu. Les deux nombres affichés (en bas à gauche, et à droite après avoir appuyé sur A) sont **les mêmes** : c'est la preuve que les deux façons s'accordent.

À quoi sert `lire()` : à savoir **ce qu'il y a à un endroit**, pas seulement où est sa propre lettre. Plus tard, c'est ainsi qu'on saura si une case est un mur, une pièce, un ennemi.

Pourquoi seulement quand on appuie sur A : regarder 360 cases prend du temps. Le faire à chaque image ralentirait le jeu ; on ne le fait que quand on le demande.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)
- `#include <nombre>` → [0.8.3. La fonction nombre\(\) — écrire un nombre](#)
- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **709 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	66 o	← la console, à l'allumage — la mise en route de la console
main()	257 o	← démarrage — ta fonction
VBlank	8 o	← la console, à chaque image — compte les images, 60 fois par seconde
AttendreImage	28 o	← image() — attend l'image suivante
AttendreVBlank	16 o	← démarrage — attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases, EcrireNombre — attend de pouvoir écrire une case

AdresseCase	10 o	← effacer(), poser(), lire()	– trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer()	– efface des cases
EcrireNombre	33 o	← nombre()	– écrit un nombre en chiffres
CopierTuiles	40 o	← démarrage	– copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage	– vide les deux cartes de l'écran
LireManette	37 o	← bouton()	– lit les huit boutons
Tuiles	176 o	← CopierTuiles	– 11 lettres sur 43

Lettres gravées : A0123456789

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

nombre()	× 4	40 o sur place, dans main()	+ EcrireNombre (33 o)
bouton()	× 5	25 o sur place, dans main()	+ LireManette (37 o)
effacer()	× 1	15 o sur place, dans main()	+ AdresseCase (10 o), EffacerCases (12 o)
poser()	× 1	13 o sur place, dans main()	+ AdresseCase (10 o)
lire()	× 1	11 o sur place, dans main()	+ AdresseCase (10 o)
image()	× 1	3 o sur place, dans main()	+ AttendreImage (28 o)

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// x, y, attente, trouveX, trouveY : voir l'onglet variables.h

#include <effacer>    // efface des cases, ou tout le fond
#include <bouton>     // lit un bouton de la manette
#include <nombre>     // écrit un nombre en chiffres
#include <poser>      // pose une tuile sur une case du fond
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <lire>       // lit la tuile posée sur une case
#include "variables.h"
```

2. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    while (true) {
        image();

        if (attente > 0) attente--;

        if (attente == 0) {
            effacer(x, y, 1);
            if (bouton(DROITE) && x < 19) { x++; attente = 8; }
            if (bouton(GAUCHE) && x > 0) { x--; attente = 8; }
            if (bouton(BAS) && y < 17) { y++; attente = 8; }
            if (bouton(HAUT) && y > 0) { y--; attente = 8; }
        }

        nombre(0, 17, x);                // ce que le programme RETIENT : x...
        nombre(4, 17, y);                // ...et y (le 0.68)
        poser(x, y, ALPHABET[0]);

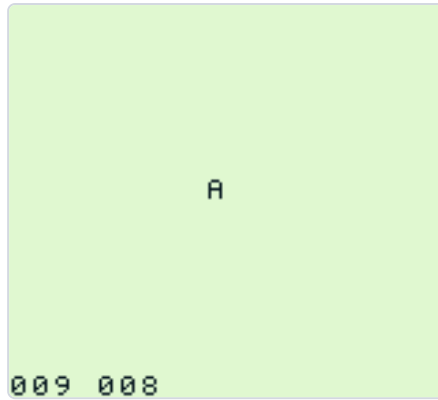
        // Le bloc ajouté : quand on appuie sur A, on CHERCHE la lettre sur l'écran.
        if (bouton(A)) {
            for (uint8_t l = 0; l < 18; l++) {          // chaque ligne, de 0 à 17...
                for (uint8_t c = 0; c < 20; c++) {      // ...et dans chaque ligne, chaque colonne
                    // lire(c, l) : la tuile affichée dans la case (c, l).
                    //
                    // if (lire(c, l) == ALPHABET[0])
                    //     |   |   |   |
                    //     |   |   |   +-- la tuile du A
                    //     |   +--+----- la case regardée : colonne c, ligne l
                    //     +----- rend ce que l'écran montre là
                    if (lire(c, l) == ALPHABET[0]) {
                        trouveX = c;                // trouvé : on retient la colonne...
                        trouveY = l;                // ...et la ligne
                    }
                }
            }
            nombre(10, 17, trouveX);                // ce que lire() a TROUVÉ, à droite :
            nombre(14, 17, trouveY);                // les mêmes nombres que x et y
        }
    }
}
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



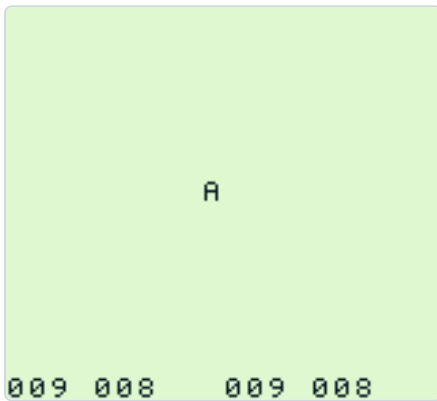
la toute première image — la console vient de démarrer — image 1



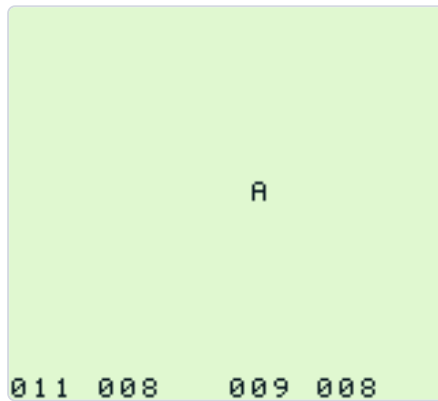
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

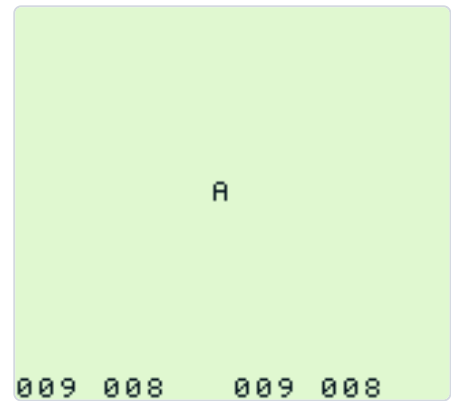
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



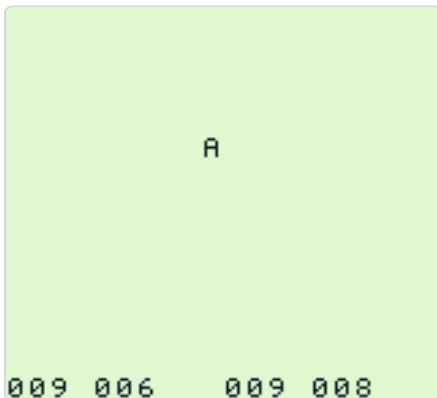
A — l'écran a changé



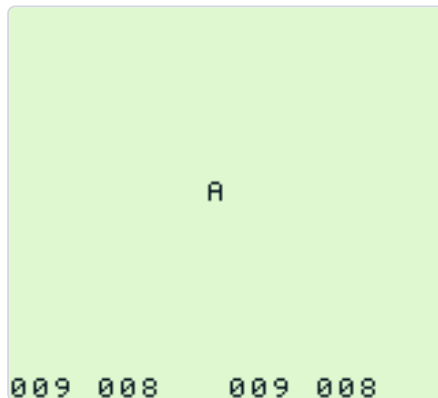
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

Le A se déplace avec les flèches. Quand on appuie sur A, sa position trouvée sur l'écran s'affiche à droite, en bas : la même que celle de gauche.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

✓ le A a bougé ($x = 13$, $y = 9$)

✓ lire() l'a trouvé à la bonne colonne ($trouveX = 13$)

✓ et à la bonne ligne ($trouveY = 9$)

✓ les deux affichages s'accordent (« 013 009 013 009 »)

Ensuite : **0.69.1. Chercher le A sur l'écran : lire — de base, ailleurs** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++